



University of
Nottingham
UK | CHINA | MALAYSIA

Fundamentals of Artificial Intelligence

Session 4: Heuristic Search | AE1FAI / COMP1037

Dr Qian Zhang | University of Nottingham Ningbo China | Spring

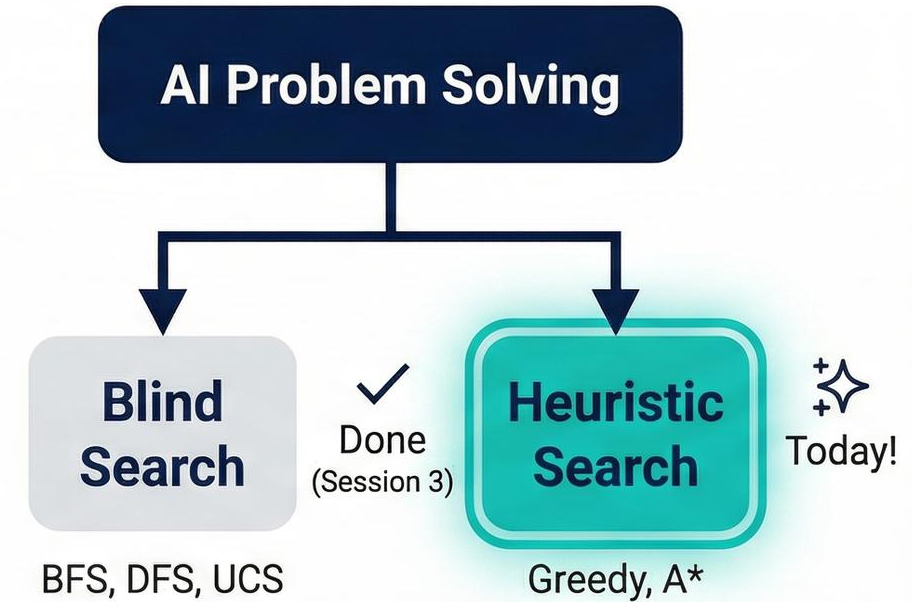


- **Recap: The Story So Far**

- Session 2: Problem Formulation & Representation
- Session 3: Uninformed Search (BFS, DFS, UCS)

- **Today: Session 4 – Heuristic Search**

- Using domain knowledge to guide search
- Algorithms: **Greedy Search, A* Search**



Learning Objectives

- Understand the concept of **informed search** and **heuristic functions**
- Implement and evaluate: **UCS**, **Greedy Search**, **A* Search**
- Evaluate heuristic functions: **admissibility** and **dominance**



Search Strategies

From blind to informed search

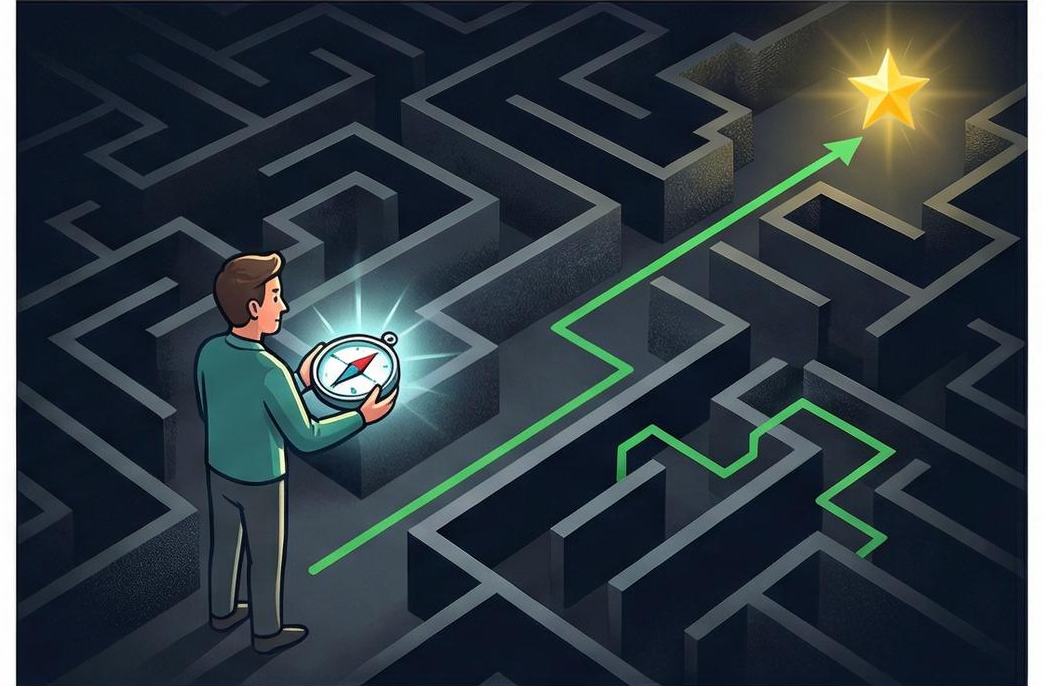


Blind Search vs. Heuristic Search



Blind Search

Explores ALL directions equally
No knowledge of where the goal is



Heuristic Search

Explores the node which is more likely
to lead to the goal state through estimation

Heuristics: educated guesses, intuitive judgments, or simply common sense

Example: Finding the Shortest Route

Problem: Find shortest route from Building A to Gate 1

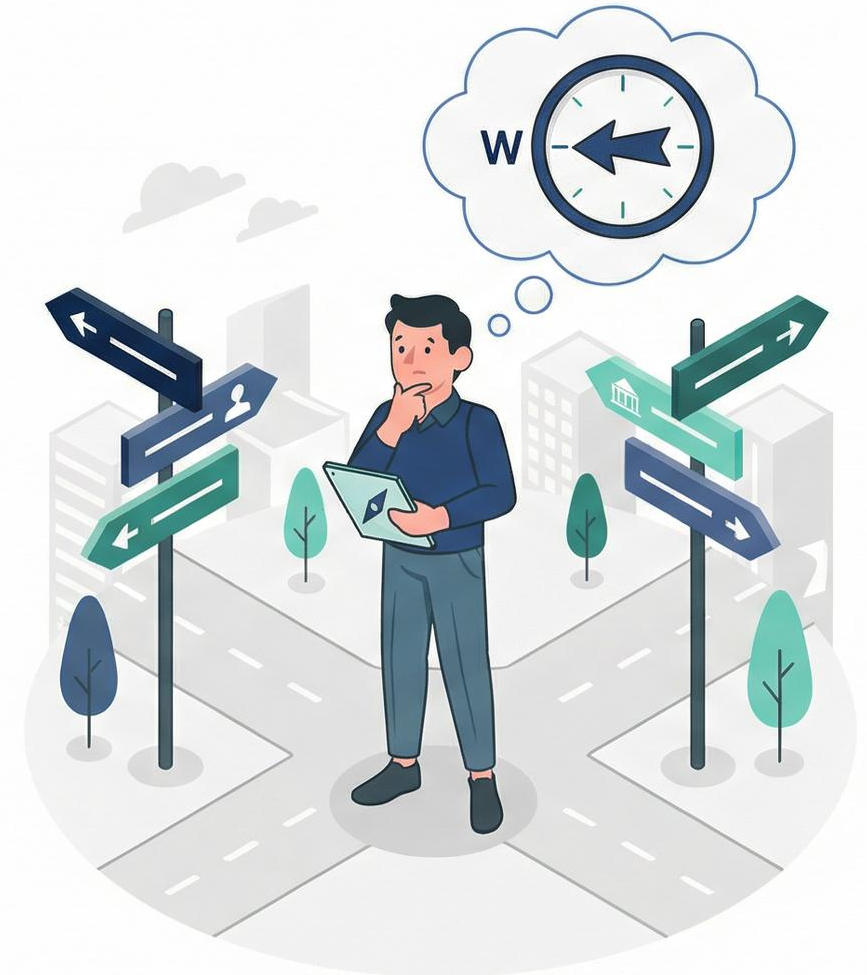
You do not have a map

Blind Search:

- Travel every road
- Record all distances
- Choose the shortest

Heuristic (Intuitive Judgment):

- Gate 1 is to the WEST
- Head west at each intersection
- Much faster decision!



Example: Finding a Book in a Library

Book: “The Secrets of the Universe”

No access to the information system

Blind Search:

- Search every floor and section
- Search every aisle and shelf
- Exhaustive but very slow

Heuristic (Common Sense):

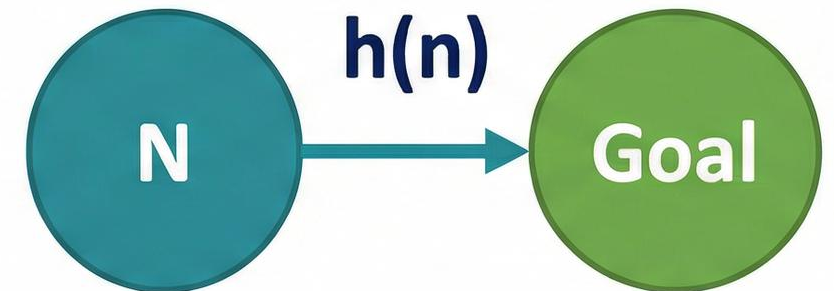
- Go to the Science section
- Look in Astronomy aisle
- Much more efficient!



- Add domain-specific information to select the best path along which to continue searching
- Sometimes known as *informed search*, it is usually more efficient than blind searches
- A heuristic method is used to **rapidly come** to a solution that is **close to the best possible answer**, or **optimal solution**



- Heuristic function $h(n)$ estimates the **goodness** of a node n
 - $h(n)$ = **estimation** of minimal cost path from n to a goal state
- All domain knowledge used in the search is encoded in $h(n)$, which is **computable** from the current state

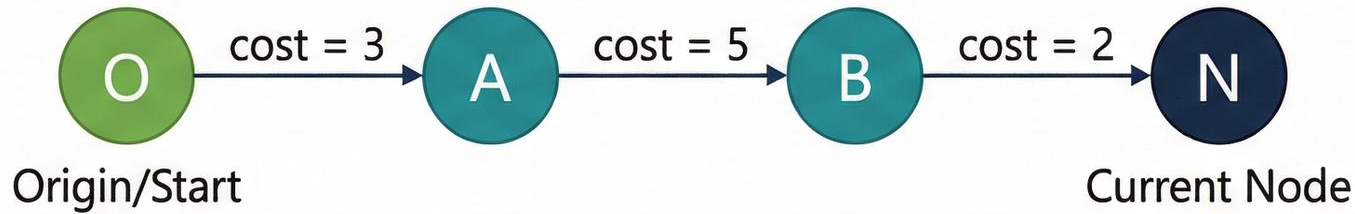


In general, $h(n) \geq 0$ for all nodes n , and $h(n) = 0$ implies that n is a goal node



Path Cost vs. Heuristic Cost

PATH COST



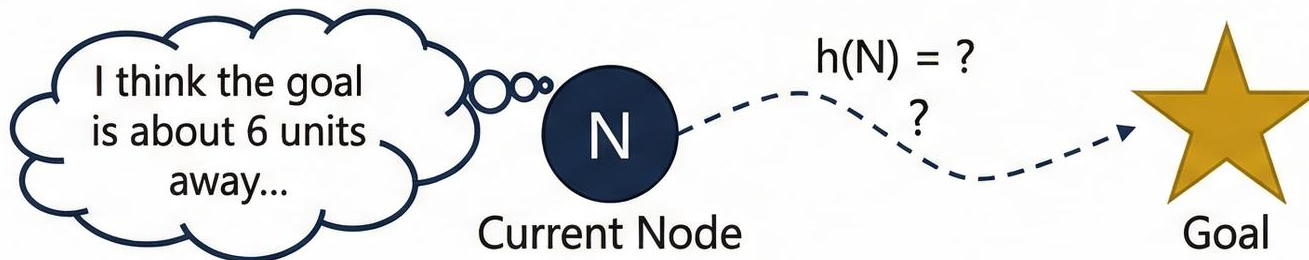
Path cost is the **ACCUMULATED REAL COST** so far

It is the actual total distance traveled from Start to N

This is a **KNOWN, EXACT** value

$$\text{Path Cost}(N) = \text{Past Cost}(O, N-1) + \text{Action Cost}(N-1, N)$$
$$\text{Path Cost}(N) = 8 + 2 = 10$$

HEURISTIC COST



Heuristic cost is an **ESTIMATION** of future cost

It is a **GUESS**, not the real cost!

Like estimating distance by looking at a map

$$\text{Heuristic Cost } h(N) = \text{Estimated future cost from } N \text{ to Goal}$$

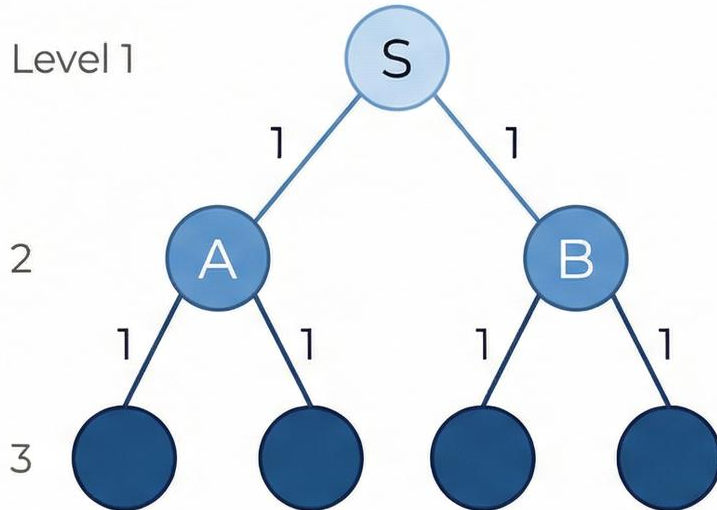
$g(n)$: Real, Accumulated, Backward-looking

$h(n)$: Estimated, Guessed, Forward-looking



Uniform Cost Search (UCS) Recap

BFS

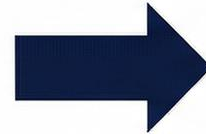


FIFO

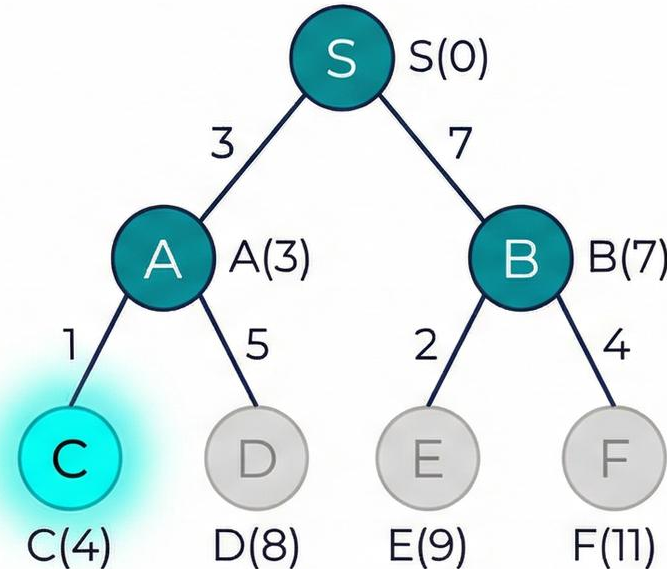
```
def enqueue_at_end(frontier, children):  
    for c in children:  
        if not is_cycle(c):  
            frontier.append(c)
```

Equal cost branches

Varying
costs?



UCS



Lowest $g(n)$ first

```
def enqueue_by_cost_g(frontier, children):  
    for c in children:  
        if not is_cycle(c):  
            frontier.append(c)  
    frontier[:] = sorted(frontier,  
                        key=lambda n: n['cost'])
```

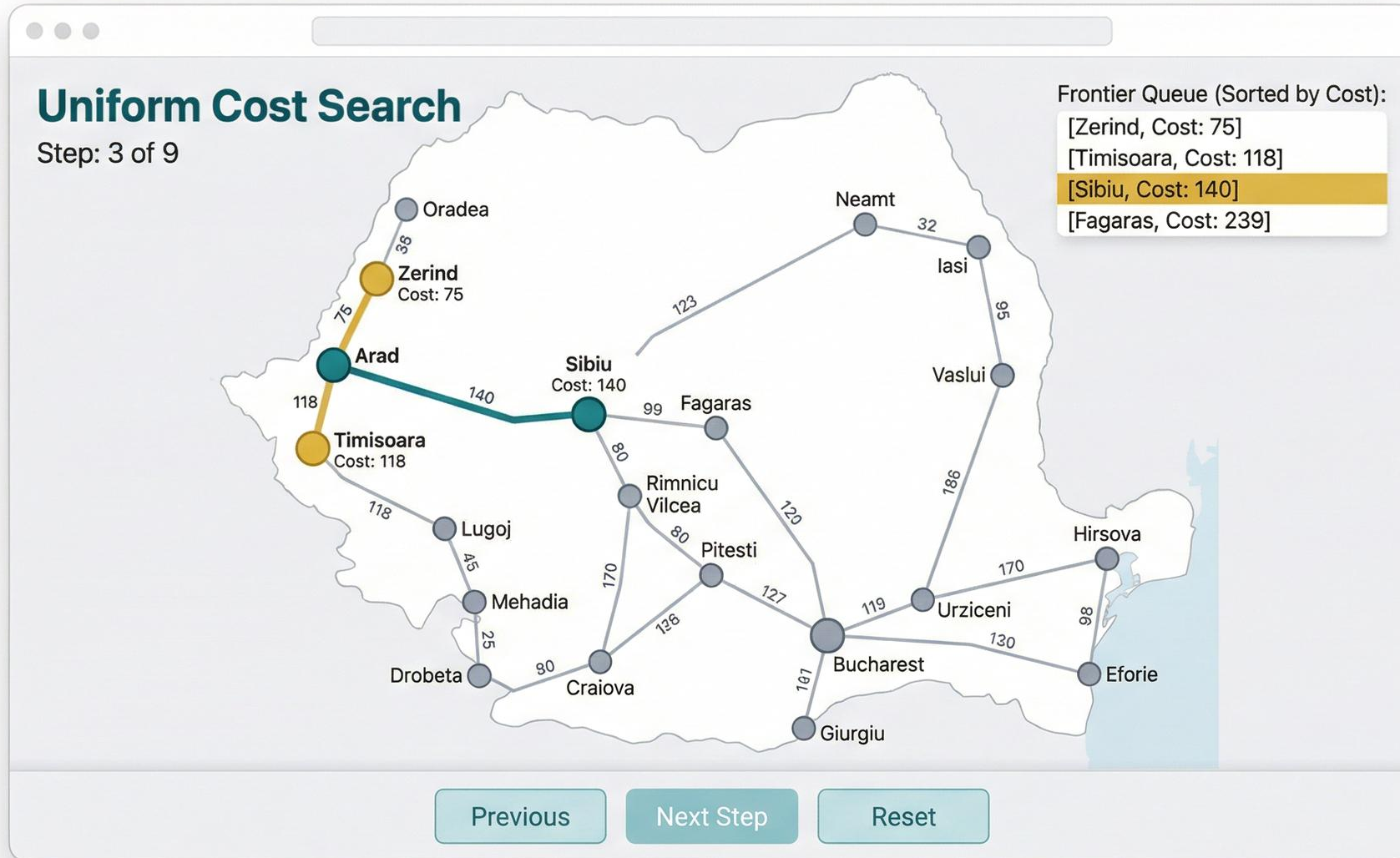
Sorts by accumulated path cost

UCS expands the
node with the
LOWEST path
cost $g(n)$

Complete and
Optimal when
all costs > 0



UCS Demo — Interactive Visualization



Click to open interactive demo [↗](#)

Greedy Search

Using heuristics to guide the search

UCS vs Greedy: Ordering the Frontier

Property	UCS	Greedy
Evaluation	$f(n) = g(n)$	$f(n) = h(n)$
Sorts by	Path cost so far	Estimated cost to goal
Optimal?	Yes ✓	No ✗
Complete?	Yes ✓	No ✗

UCS:

```
def enqueue_by_cost_g(frontier, children):  
    for c in children:  
        if not is_cycle(c):  
            frontier.append(c)  
    frontier[:] = sorted(frontier,  
                        key=lambda n: n['cost'])
```

Greedy:

```
def enqueue_by_cost_h(frontier, children):  
    for c in children:  
        if not is_cycle(c):  
            frontier.append(c)  
    frontier[:] = sorted(frontier,  
                        key=lambda n: heuristic_sld(n['state'], p))
```

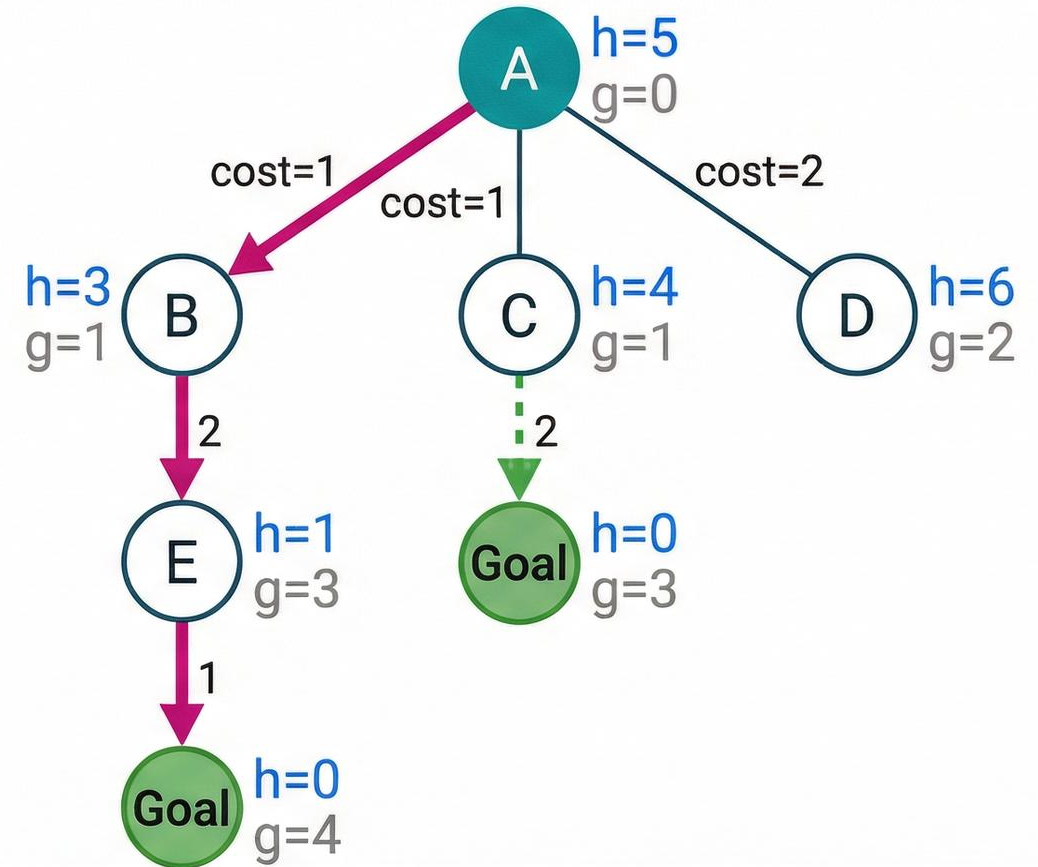

Greedy Search Algorithm

Greedy Search Strategy

- Frontier sorted by $h(n)$ only
- Always expands the node that **APPEARS** closest to the goal
- **Ignores** the actual path cost $g(n)$ completely

- 🕒 Uses only **heuristic** estimate $h(n)$
- 👁️ **Blind** to accumulated cost $g(n)$
- ⚡ **Fast but risky** — no optimality guarantee

Warning: Greedy search is NOT complete and NOT optimal



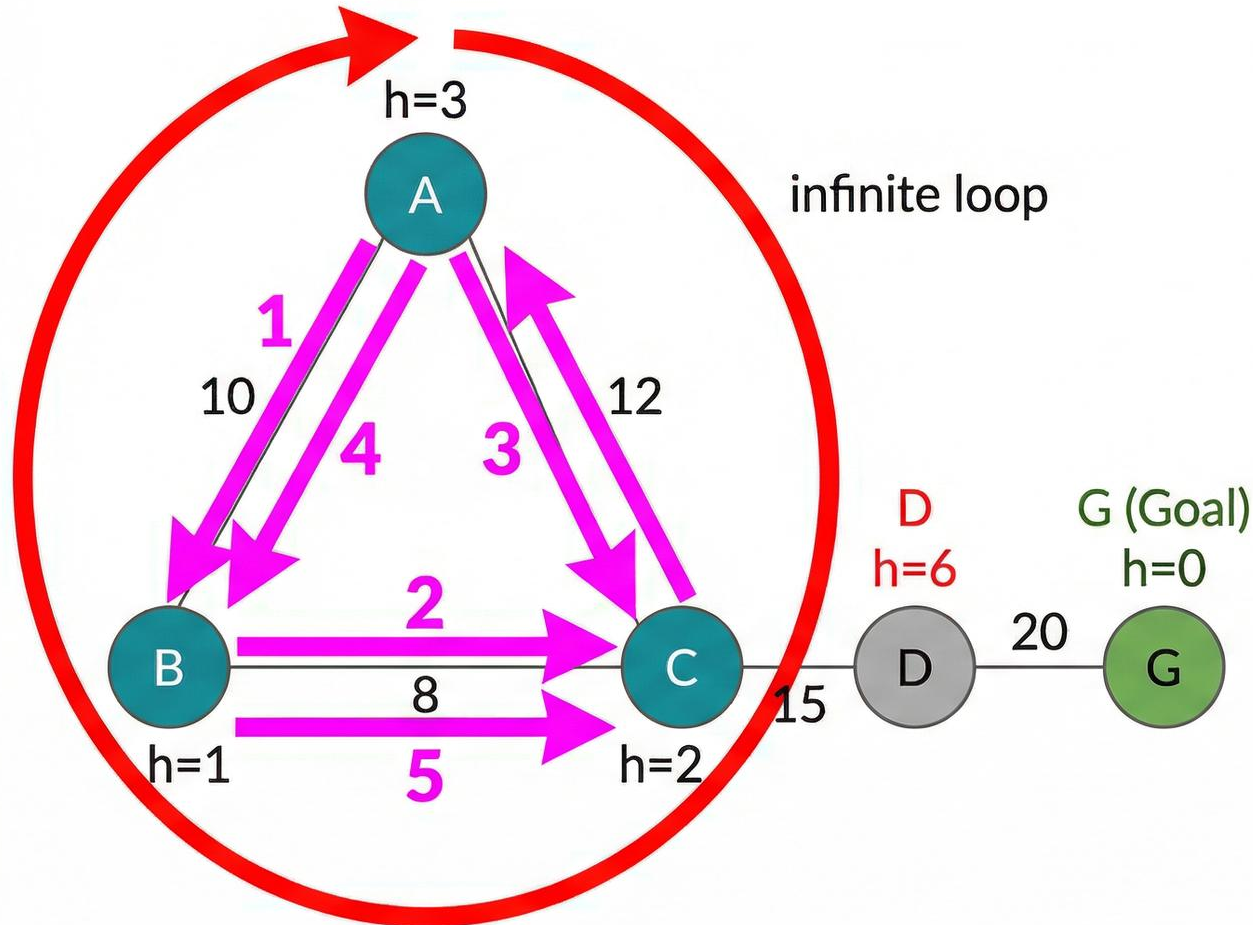
Greedy path: $A \rightarrow B \rightarrow E \rightarrow \text{Goal}$, cost = 4

Optimal path: $A \rightarrow C \rightarrow \text{Goal}$, cost = 3

Greedy chose B ($h=3$) over C ($h=4$) because $h(B) < h(C)$
But the real optimal path goes through C!



Greedy Search: Not Complete



Why It Loops

Step 1: Expand A $\rightarrow$ frontier: B($h=1$), C($h=2$) $\rightarrow$ Pick B

Step 2: Expand B $\rightarrow$ frontier: C($h=2$) [A excluded] $\rightarrow$ Pick C

Step 3: Expand C $\rightarrow$ frontier: A($h=3$), D($h=6$) [B excluded] $\rightarrow$ Pick A ($h=3 < h=6$!)

Step 4: Expand A $\rightarrow$ frontier: B($h=1$) [C excluded] $\rightarrow$ Pick B

Step 5: CYCLE: A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$... forever!

D is NEVER expanded because $h(D)=6$ is always higher than triangle nodes!

The only path to Goal (A $\rightarrow$ C $\rightarrow$ D $\rightarrow$ G) is never found.

Key Insight: Greedy tree search is NOT complete – it can loop forever through cycles

Greedy Search: Arad to Bucharest

$h_{SLD}(n)$ = straight line distance to Bucharest

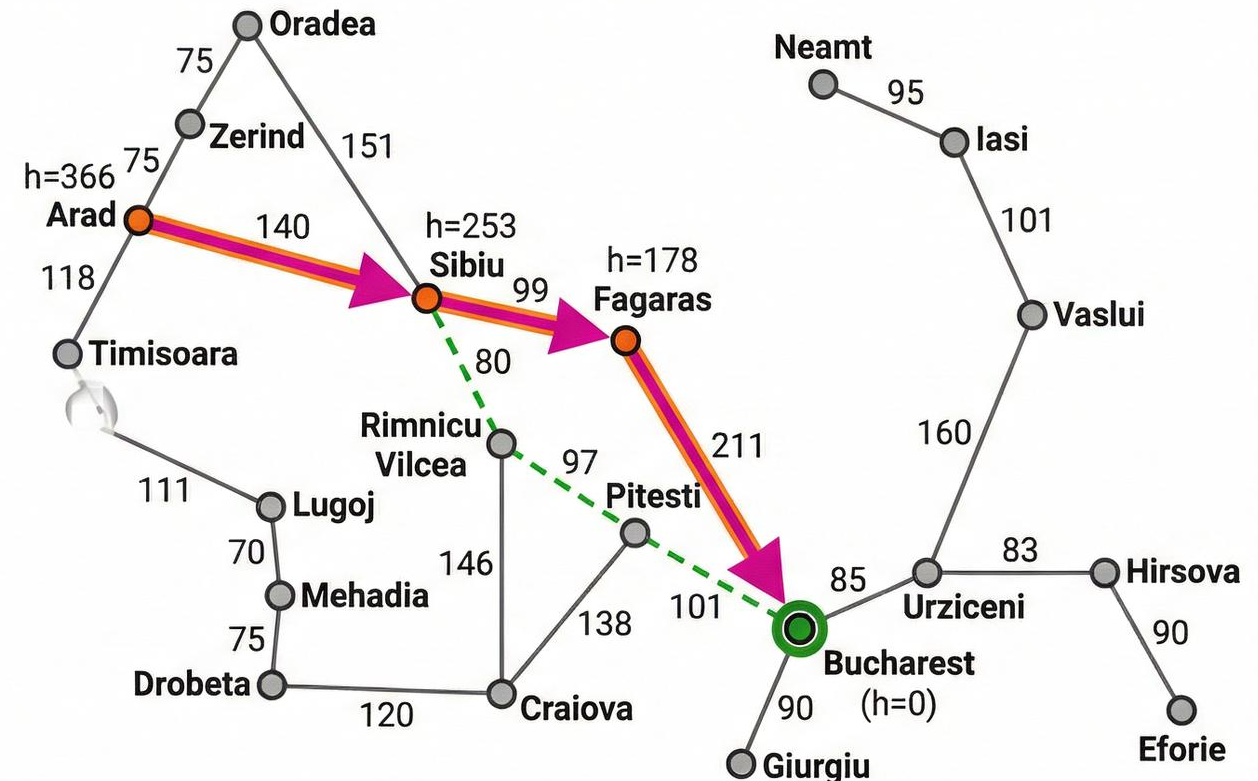
1. Arad $\rightarrow$ Sibiu (SLD=253)
2. Sibiu $\rightarrow$ Fagaras (SLD=178)
3. Fagaras $\rightarrow$ Bucharest (SLD=0)

Total cost: $140 + 99 + 211 = 450$ km

NOT OPTIMAL

Optimal path via Rimnicu-Pitesti: 418 km

Greedy search performed well but
found a suboptimal solution



Greedy Search Evaluation

✗ NOT Complete

Can loop forever (e.g.,
 $A \rightarrow B \rightarrow C \rightarrow A \rightarrow B \rightarrow C \rightarrow \dots$ triangle
cycle, $D(h=6)$ never expanded)

✗ NOT Optimal

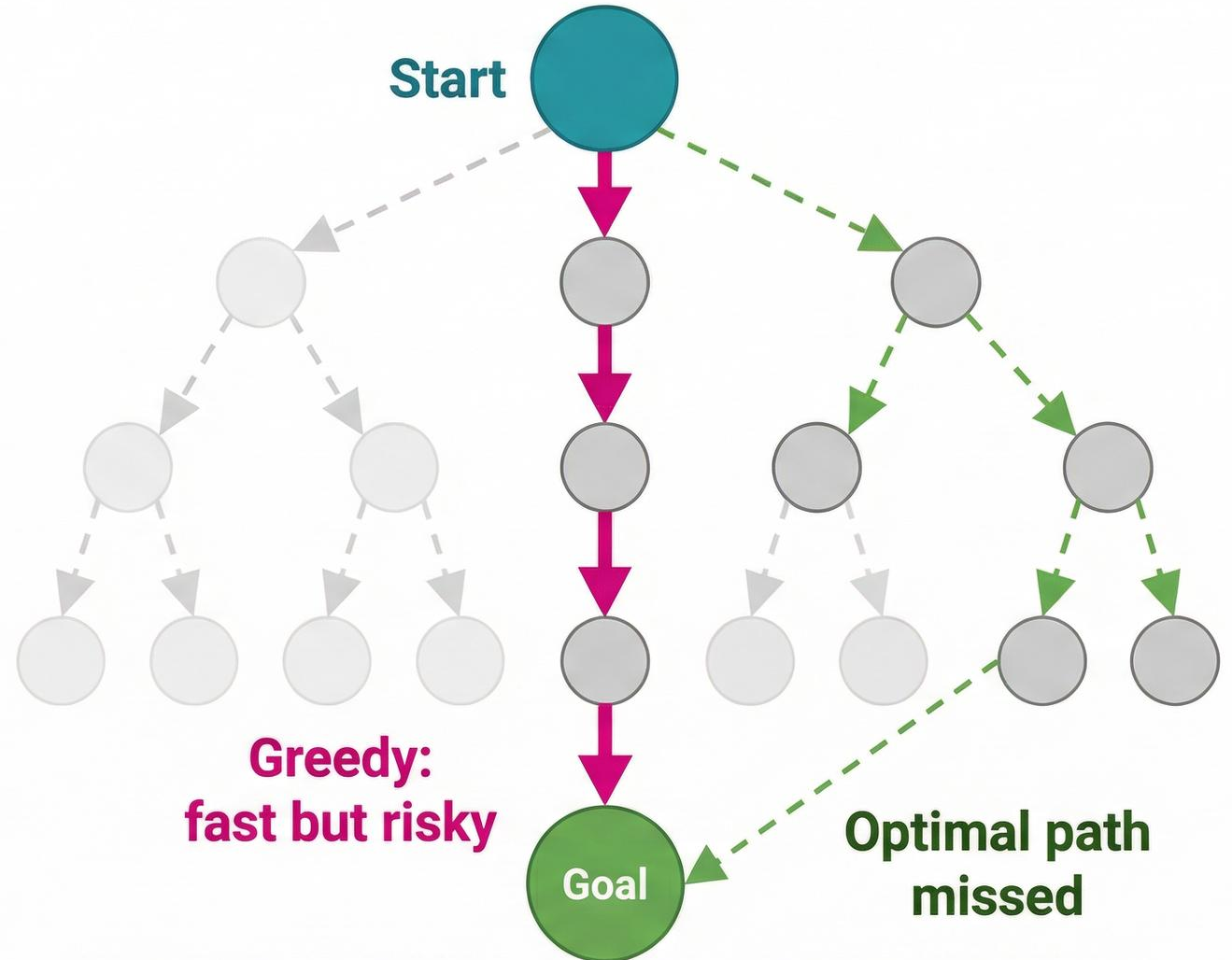
Picks lowest $h(n)$, ignores actual
path cost $g(n)$

🕒 $O(b^d)$

b = branching factor, d = depth
of solution

💾 $O(b^d)$

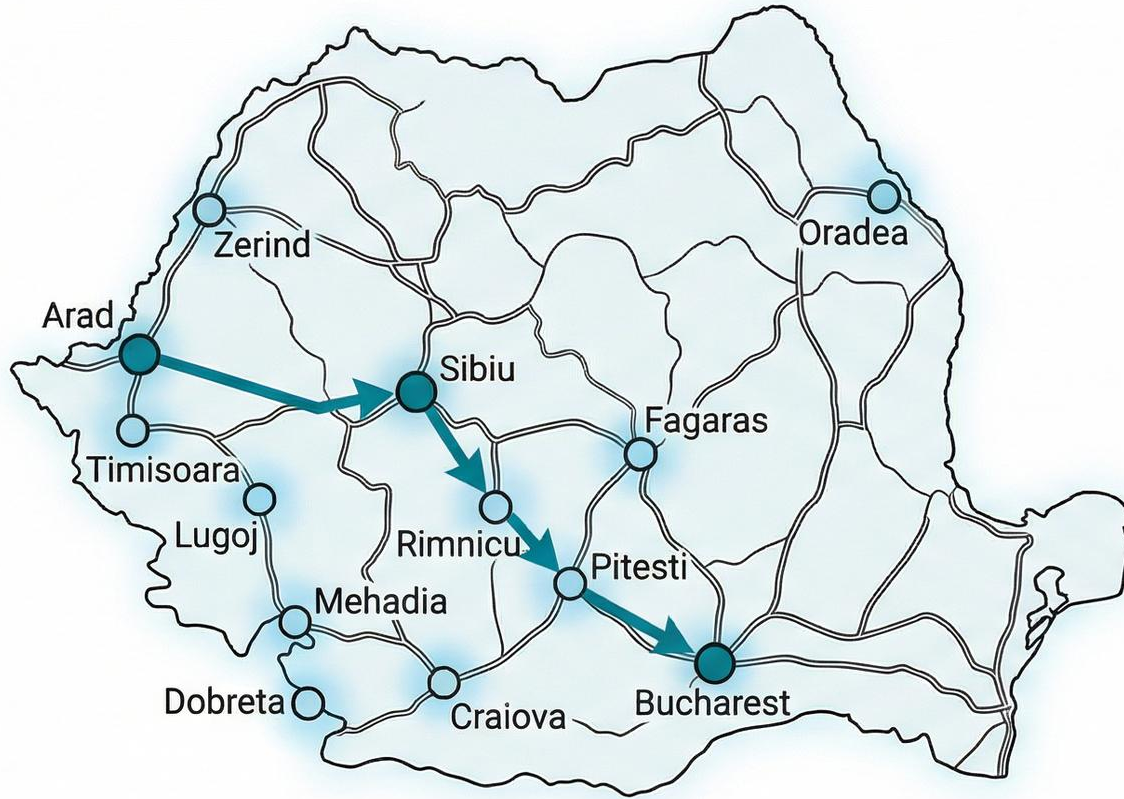
Keeps all nodes in memory



**Greedy search performed well in practice
but has NO theoretical guarantees**



UCS vs. Greedy Search on Romania Map

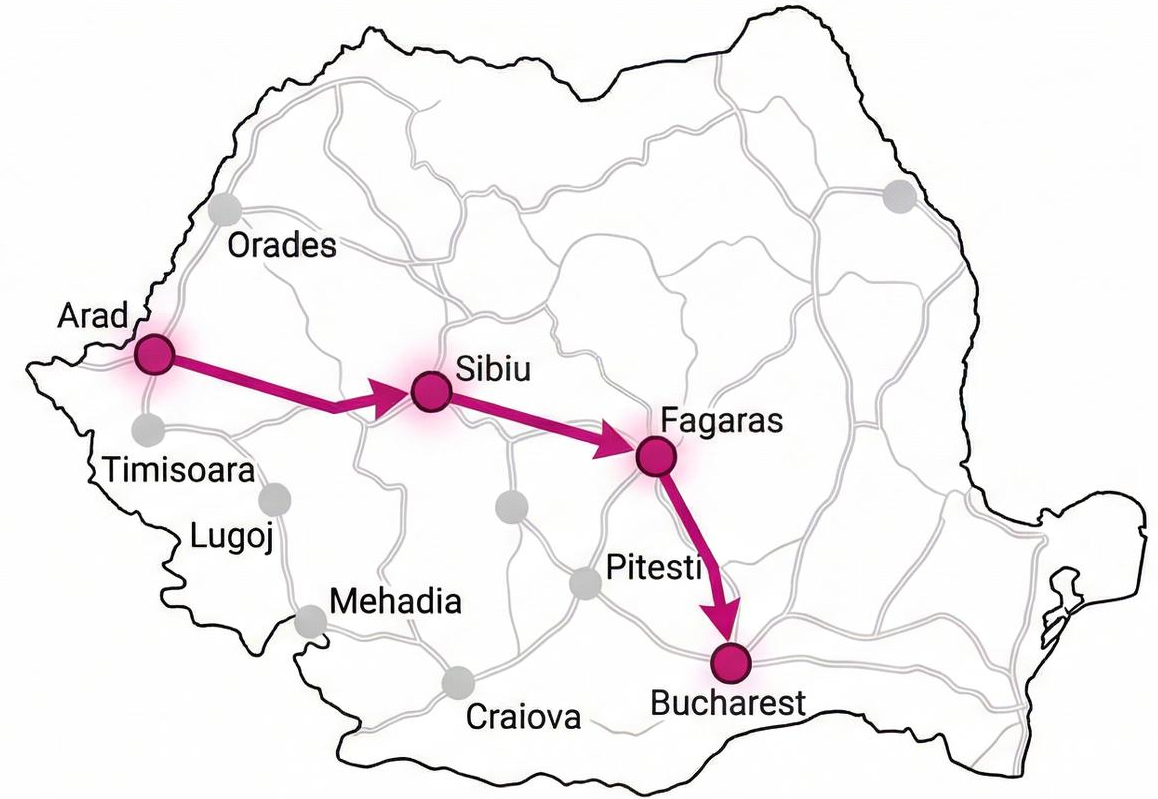


Path: **Arad** → **Sibiu** → **Rimnicu** → **Pitesti** → **Bucharest**

Cost: **418 km (OPTIMAL)** ✓

Nodes expanded: **18**

Explores widely to guarantee optimal solution



Path: **Arad** → **Sibiu** → **Fagaras** → **Bucharest**

Cost: **450 km (NOT optimal, +32 km)** ✗

Nodes expanded: **4**

Fast but misses the optimal route

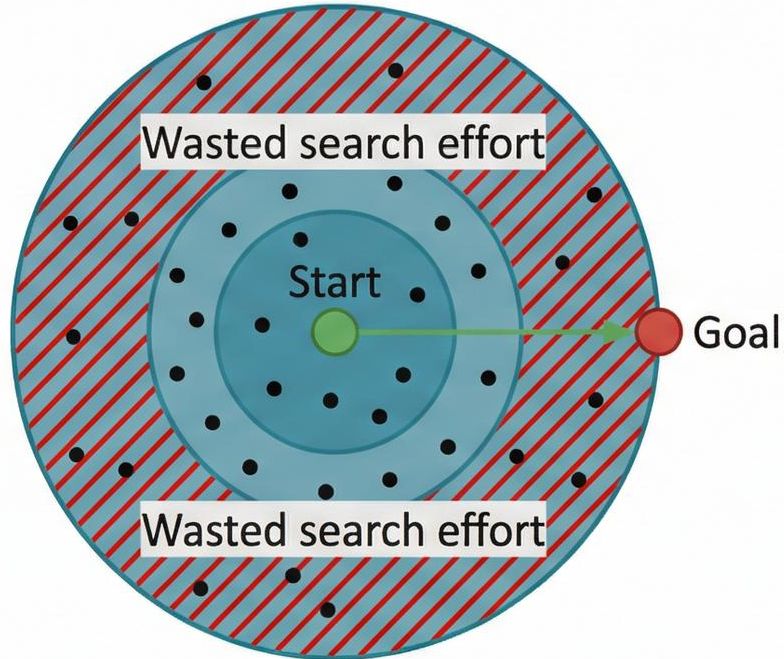
UCS: 18 nodes, 418 km, **OPTIMAL**

Trade-off: Optimality vs Speed

Greedy: 4 nodes, 450 km, **NOT** optimal

Search Domain: UCS vs. Greedy

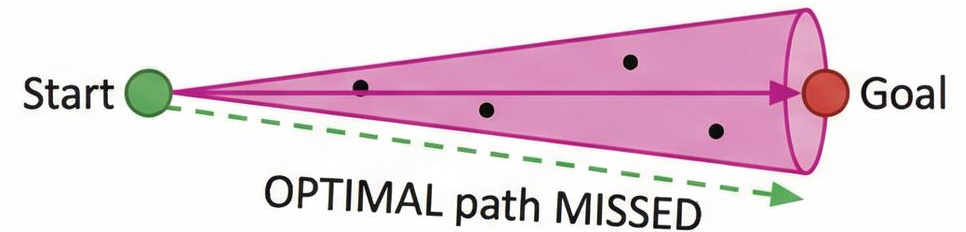
UCS Search Domain



UCS: Expands uniformly by cost

Searches EVERYWHERE — guarantees optimal
But wastes effort exploring irrelevant directions
Nodes expanded: 18

Greedy Search Domain



Greedy: Follows $h(n)$ directly toward goal

Searches very LITTLE — extremely fast
But biases too much toward $h(n)$, misses optimal solution
Nodes expanded: 4

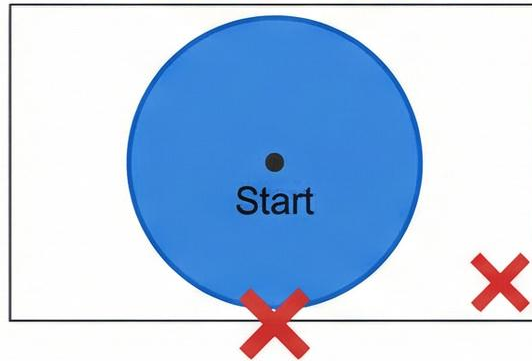
UCS explores too broadly (wasted effort) | Greedy explores too narrowly (misses optimal)

Can we combine the best of both? →

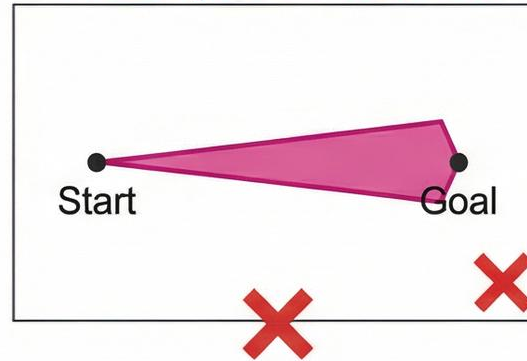


A* Search: Best of Both Worlds

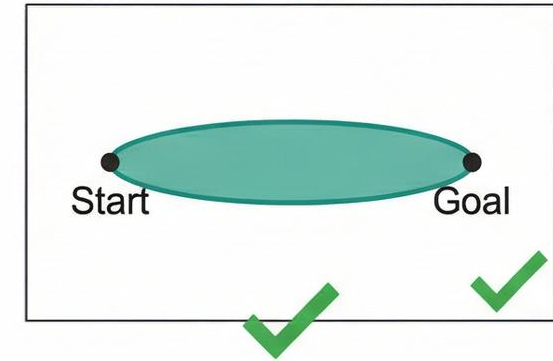
UCS: Too broad



Greedy: Too narrow



A*: Just right!



$$f(n) = g(n) + h(n)$$

$g(n)$



- Actual path cost from Start to n
- Backward-looking (real, accumulated)
- From UCS — ensures optimality



$h(n)$

- Estimated cost from n to Goal
- Forward-looking (heuristic, estimated)
- From Greedy — provides efficiency



A* = Best of Both Worlds: Optimal like UCS + Efficient like Greedy

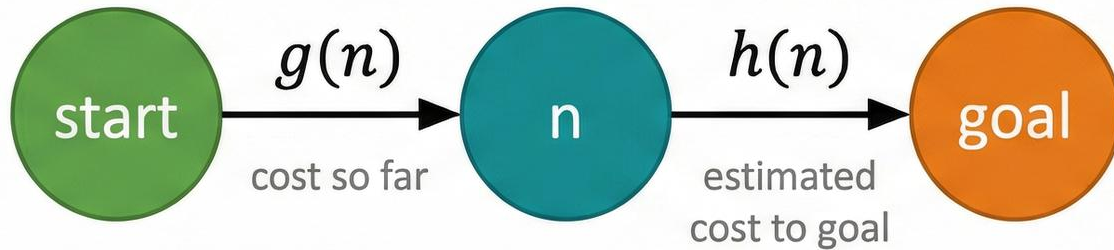
Expands the node with the lowest $f(n) = g(n) + h(n)$

Balances actual cost (past) with estimated remaining cost (future)

A* Search Algorithm

Combining path cost and heuristic estimation

A* Algorithm: $f(n) = g(n) + h(n)$



$$f(n) = g(n) + h(n)$$

Estimated cost of cheapest solution through n

- $g(n)$: actual cost from start to n
- $h(n)$: estimated cost from n to goal
- $h=0 \rightarrow$ A* becomes **UCS** (complete & optimal)
- $g=0 \rightarrow$ A* becomes **Greedy** (lose optimality)

```
1 def enqueue_by_cost_gh(frontier, children):
2     for c in children:
3         if not is_cycle(c):
4             frontier.append(c)
5     frontier[:] = sorted(frontier,
6                           key=lambda n: n['cost']
7                           + heuristic_sld(n['state'], p))
```

$f(n) = g(n) + h(n)$

Combines UCS (g) + Greedy (h)

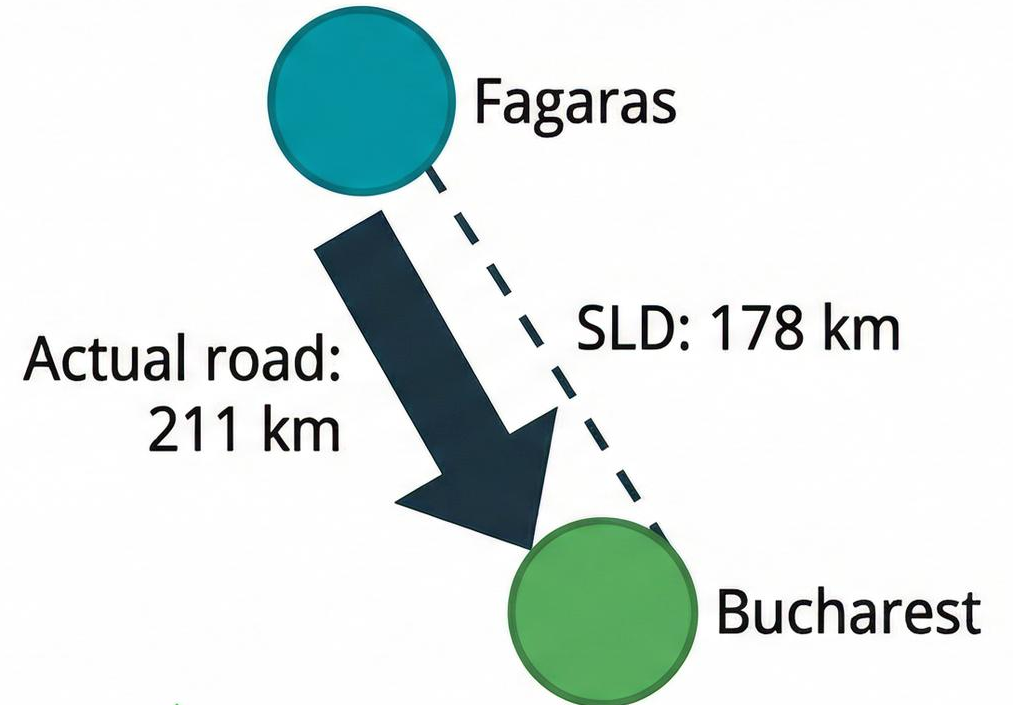
A* is optimal and complete
when $h(n)$ is admissible

How to estimate h ?

A* is optimal and complete if the heuristic is admissible

Admissible heuristic:

- The heuristic must never overestimate the cost to reach the goal
- $h(n)$ is a valid lower bound on cost to the goal



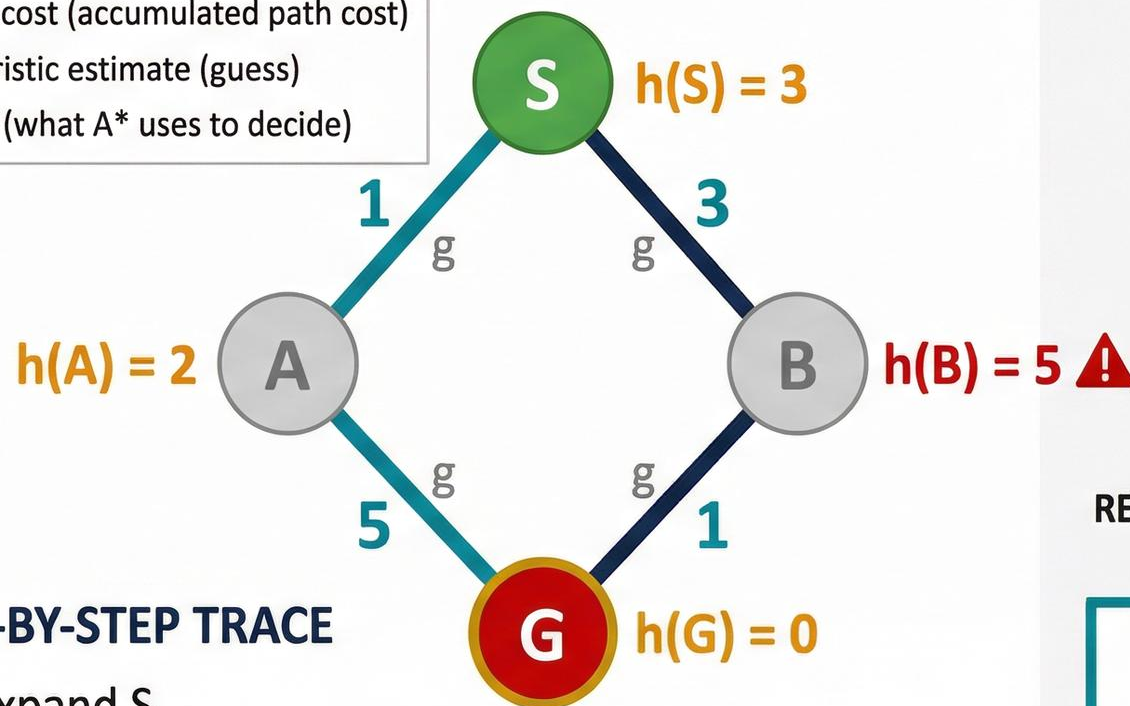
✓ Admissible! $211 \geq 178$
 $g(F,B) = 211 \geq h(F,B) = 178$

SLD is admissible because road distance is always $\geq$ straight-line distance



Why Admissibility Matters

- g = Real cost (accumulated path cost)
- h = Heuristic estimate (guess)
- $f = g + h$ (what A* uses to decide)



A* STEP-BY-STEP TRACE

Step 1: Expand S

Children: A($g=1, h=2, f=3$), B($g=3, h=5, f=8$)

A* picks A ($f=3 < f=8$) ← lowest f

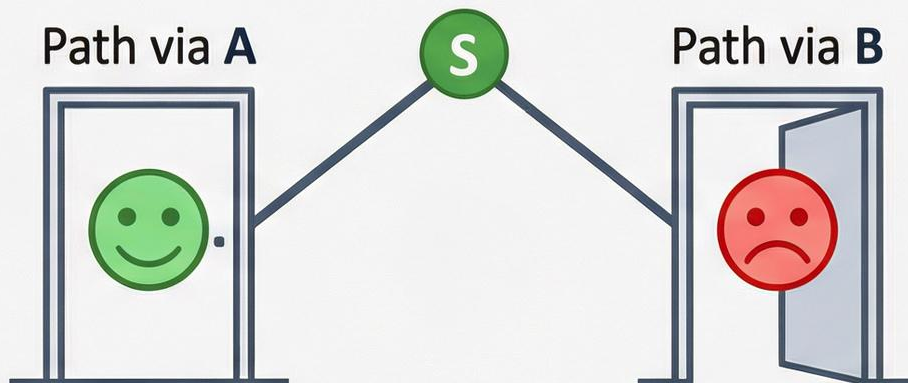
Step 2: Expand A

Children: G($g=1+5=6, h=0, f=6$)

Frontier: B($f=8$), G($f=6$). A* picks G ($f=6$) ← GOAL FOUND!

Result: $S \rightarrow A \rightarrow G$, total cost = 6 ❌ **NOT OPTIMAL**

Optimal: $S \rightarrow B \rightarrow G$, total cost = $3+1 = 4$ ✅ **MISSED!**



$h=2, f=3$ — Looks cheap!

$h=5, f=8$ — Looks expensive!

REALITY: Total cost = 6 (expensive!)

REALITY: Total cost = 4 (cheap!)

$h(B) = 5$ OVERESTIMATES (real cost to goal = 1)

This makes **B** look expensive: **$f(B) = 3 + 5 = 8$**

So **A*** avoids **B** and picks **A** instead

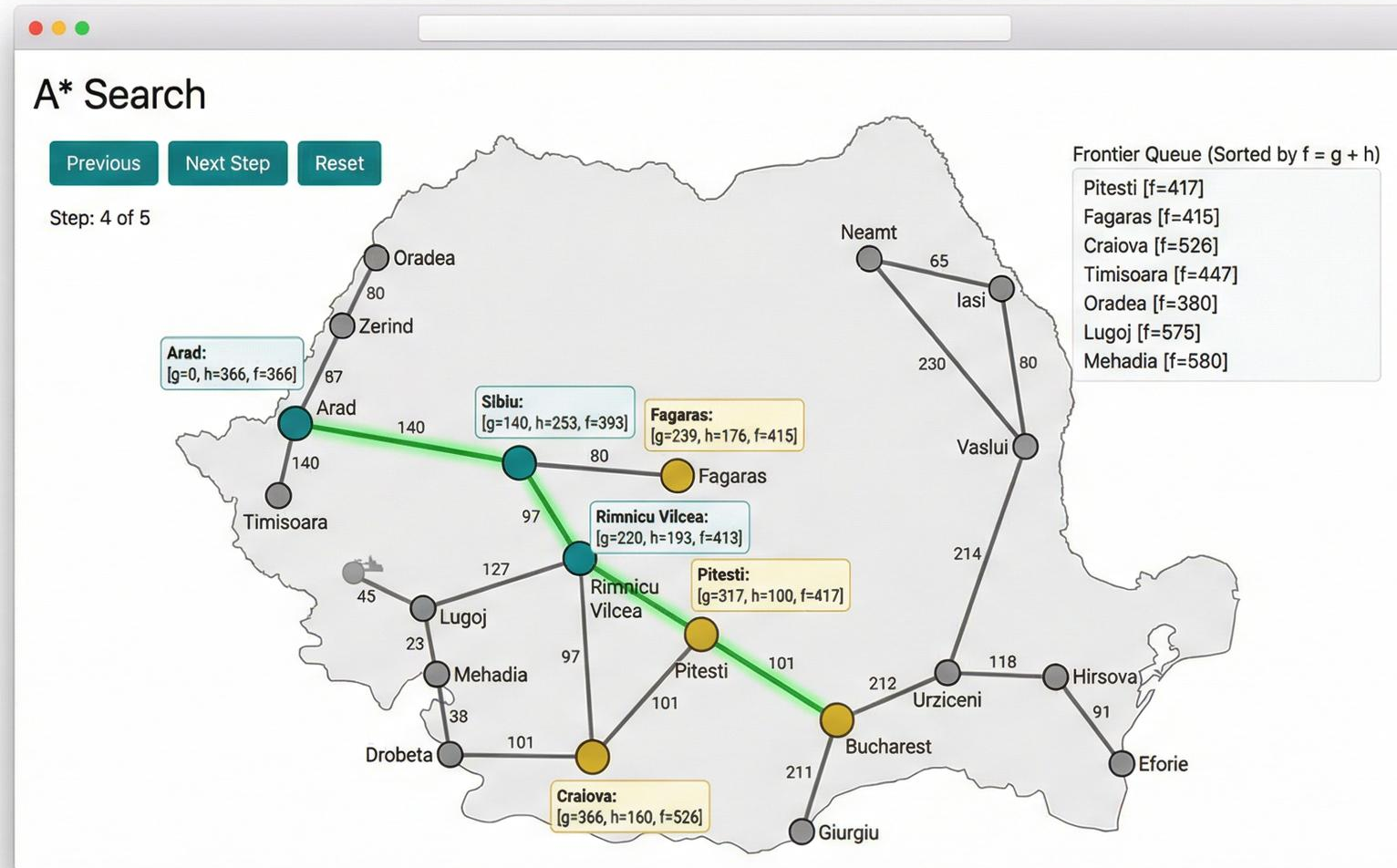
But **A** actually leads to a **MORE** expensive path!

✅ **Admissible:** $h(n) \leq h^*(n) \rightarrow$ A* guaranteed optimal

❌ **Non-admissible:** $h(n) > h^*(n) \rightarrow$ A* may miss optimal



A* Demo — Interactive Visualization



[Click to open interactive demo](#)



A* Search Tree

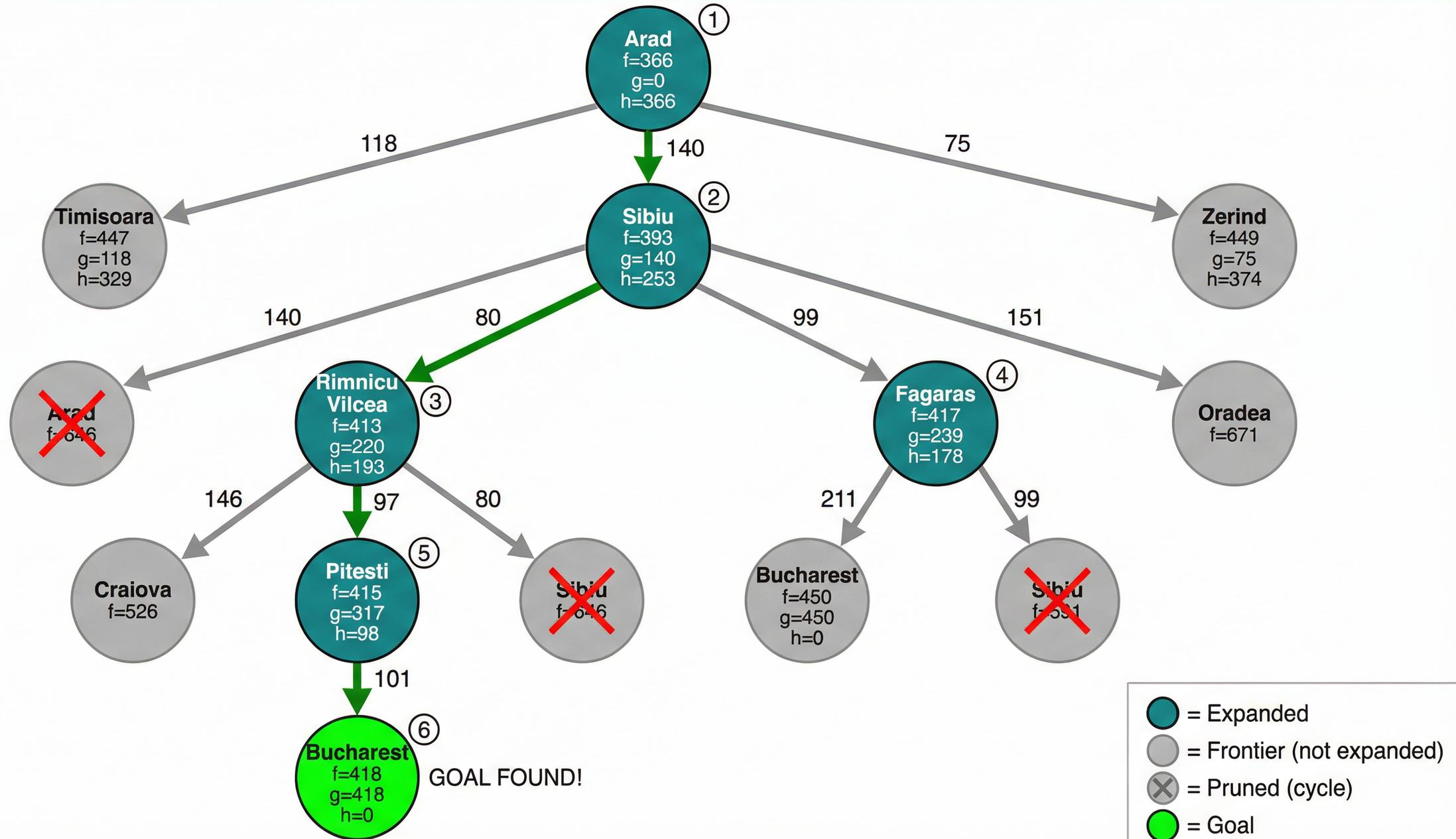
Level 0

Level 1

Level 2

Level 3

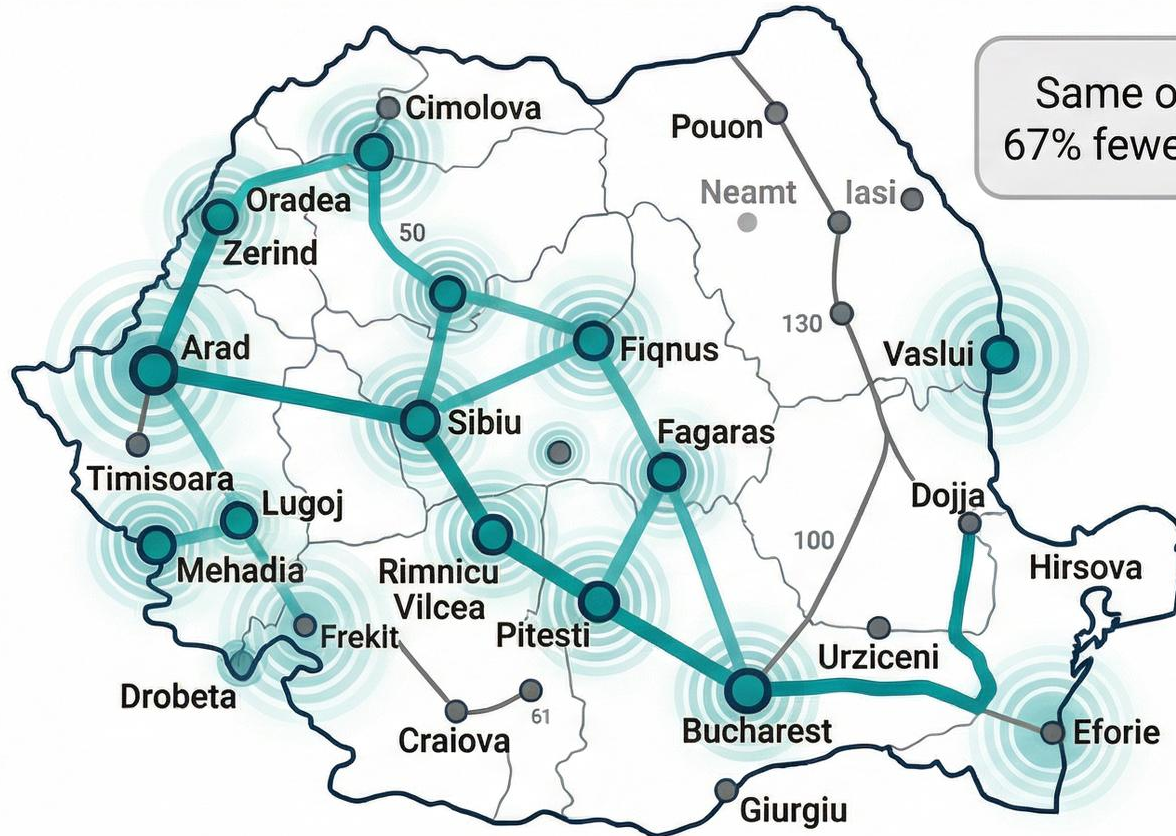
Level 4





UCS vs. A* Comparison

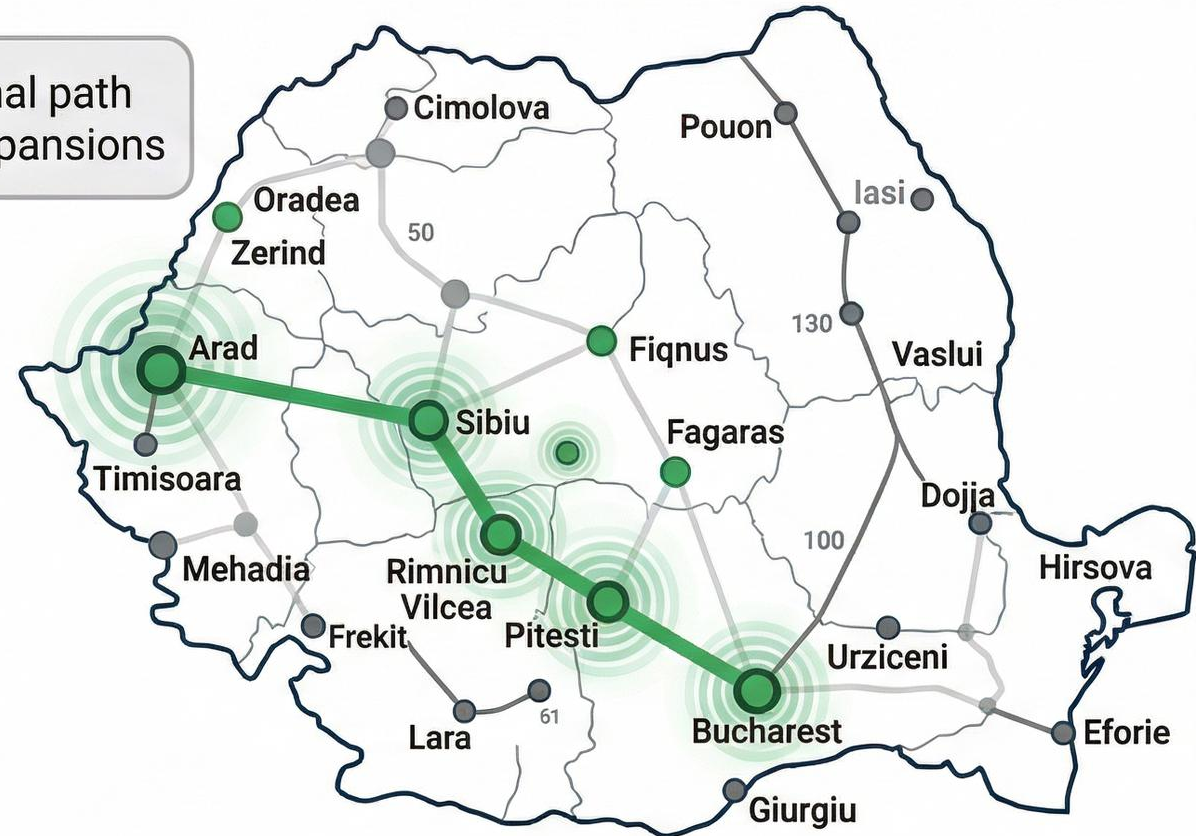
UCS



- EXPANDED NODES
- UNEXPLORED NODES
- OPTIMAL PATH

Expansions: 18
Cost: 418 km ✓ Optimal
Explores too many nodes

A*



- EXPANDED NODES
- UNEXPLORED NODES
- OPTIMAL PATH

Expansions: 6
Cost: 418 km ✓ Optimal
Much more efficient!

Same optimal path
67% fewer expansions

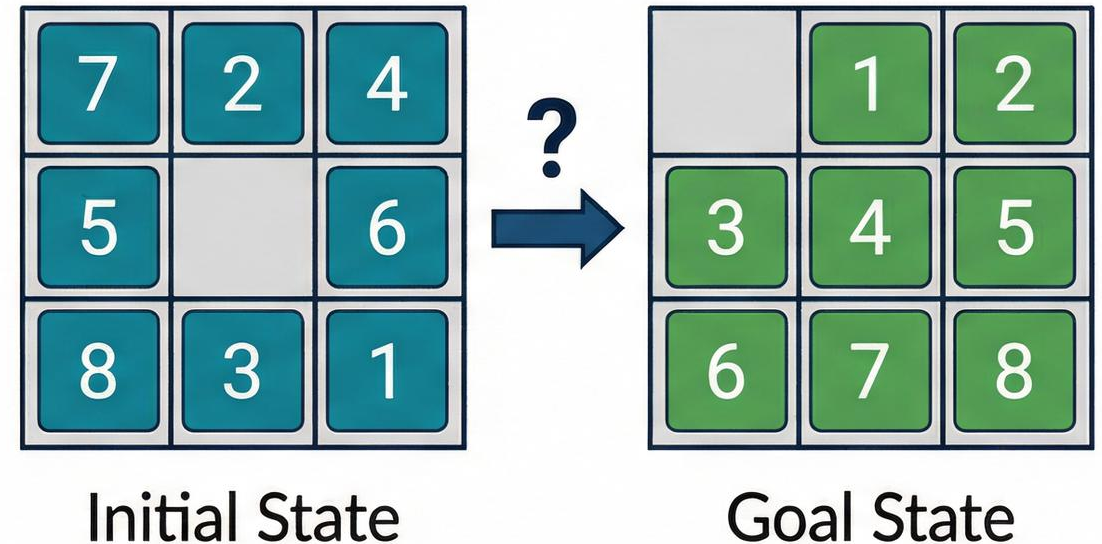
Heuristic Function Design

Using the 8-Puzzle as a case study

8-Puzzle: A Classic AI Problem

- 3x3 grid with 8 numbered tiles and 1 blank space
- Tiles can slide into the blank space
- **Goal:** reach a specific target configuration
- State space: $9!/2 = 181,440$ reachable states

Perfect testbed for **comparing search algorithms** and **heuristics**





8-Puzzle Demo — Interactive Visualization

Algorithm: A* with Manhattan Distance

Step: 12 of 26

Current State: (3, 1, 2; 4, 0, 5; 6, 7, 8)	h value (Manhattan): 6	g value (Cost so far): 12	f value (g + h): 18
---	---------------------------	------------------------------	------------------------

[Click to open interactive demo](#)



h1: Misplaced Tiles vs. h2: Manhattan Distance

2	8	3
1	6	4
7		5

Current State



1	2	3
8		4
7	6	5

Goal State

h1: Misplaced Tiles

1	1	0
1	1	0
0	-	0

Blank tile NOT counted!

$$h1 = 0+1+1+0+0+1+0+1 = 4$$

h1 = 4

h2: Manhattan Distance

1	2	0
1	1	0
0	-	0

$$h2 = 1+1+0+0+0+1+0+2 = 5$$

h2 = 5

2	8	3
1	6	4
7		5

Blank tile NOT counted!
h1 = 1+1+0+0+1+0+1 = 4

h1 = 4

WHY h2 >= h1 (visual proof)

Tile	1	2	3	4	5	6	7	8
h1 row	1	1	0	0	0	1	0	1
h2 row	1	1	0	0	0	1	0	2

→ SUM = 4

→ SUM = 5

KEY INSIGHT:

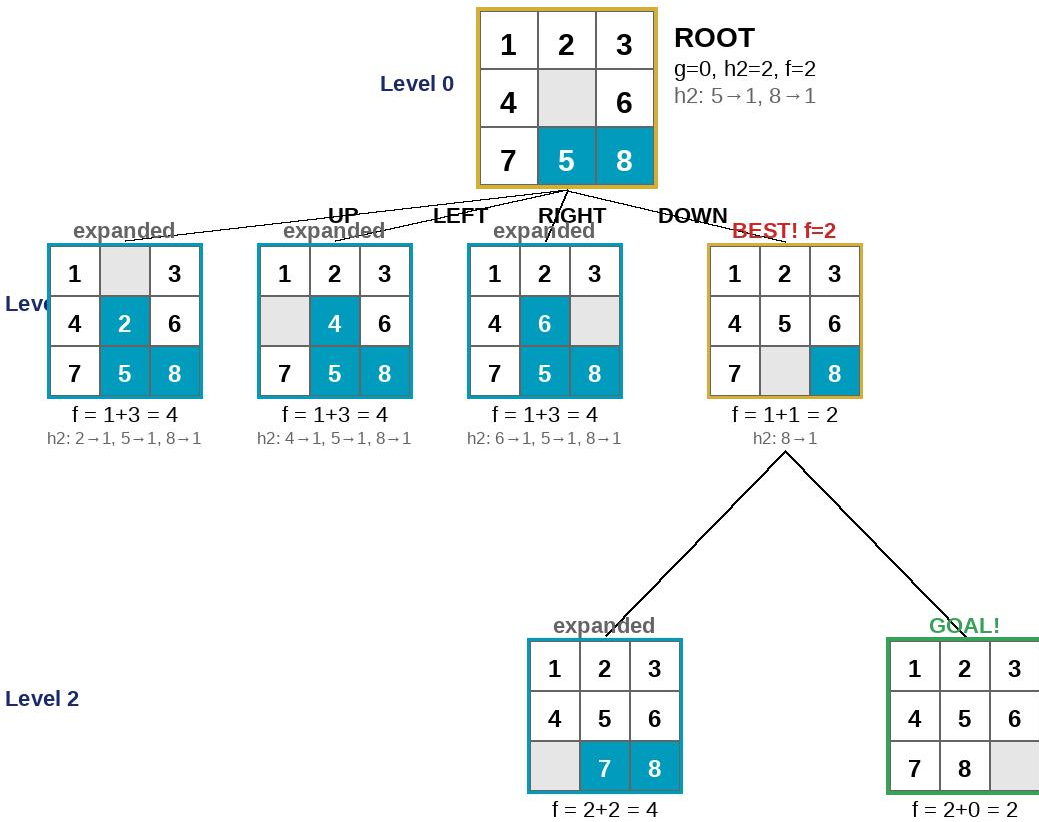
For each tile: if misplaced, Manhattan distance >= 1 (at least 1 move needed)

So h2(tile) >= h1(tile) for EVERY tile → h2 >= h1 **ALWAYS**

h2 DOMINATES h1: h2 is more informed



A* with h2: Manhattan Distance



Goal State

1	2	3
4	5	6
7	8	

A* with h2 (Manhattan Distance)

- h2 gives HIGHER estimates than h1
- Fewer nodes have similar f values
- A* expands FEWER nodes
- h2 is more DISCRIMINATING

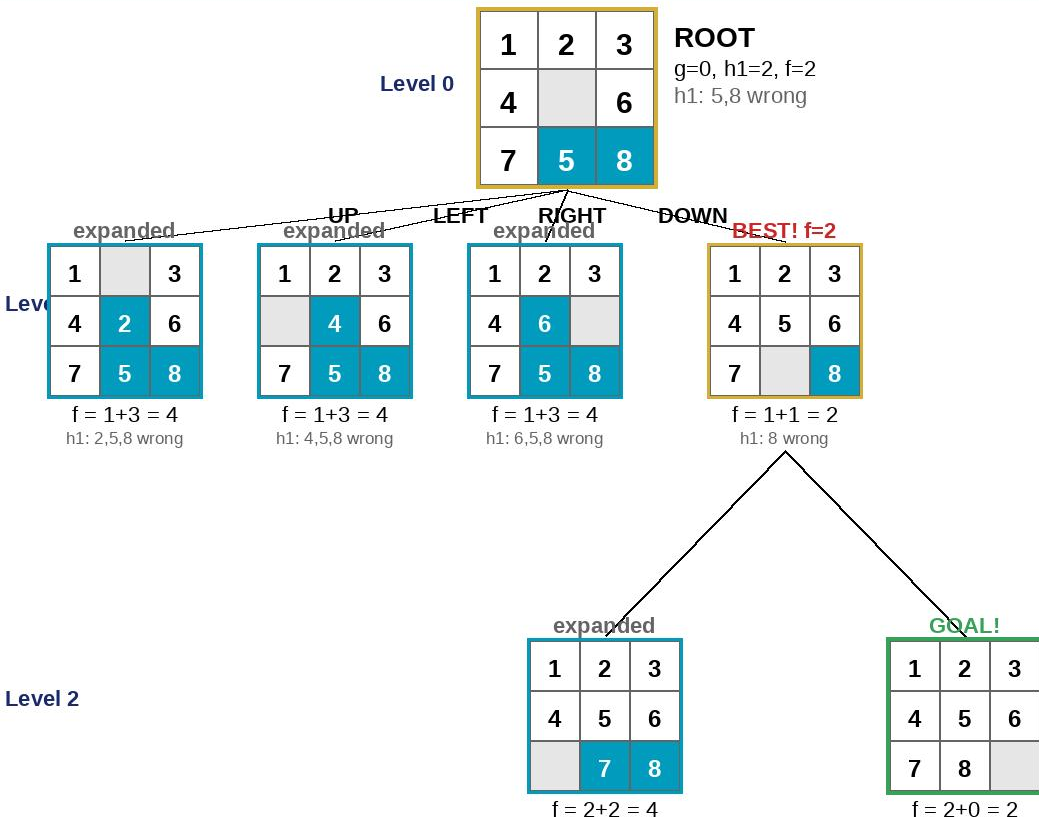
**h2 expanded only 2 nodes
to find goal**

Legend:

-  = Misplaced tile (wrong position)
-  = Correct tile / Blank tile
-  = Selected node (lowest f)



A* with h1: Misplaced Tiles



Goal State

1	2	3
4	5	6
7	8	

A* with h1 (Misplaced Tiles)

- h1 gives LOWER estimates than h2
- More nodes have SIMILAR f values
- A* must expand MORE nodes
- h1 is LESS discriminating

For this simple example, h1 = h2.

But for harder puzzles:

h1 expanded ~12 nodes
vs h2 expanded ~5 nodes

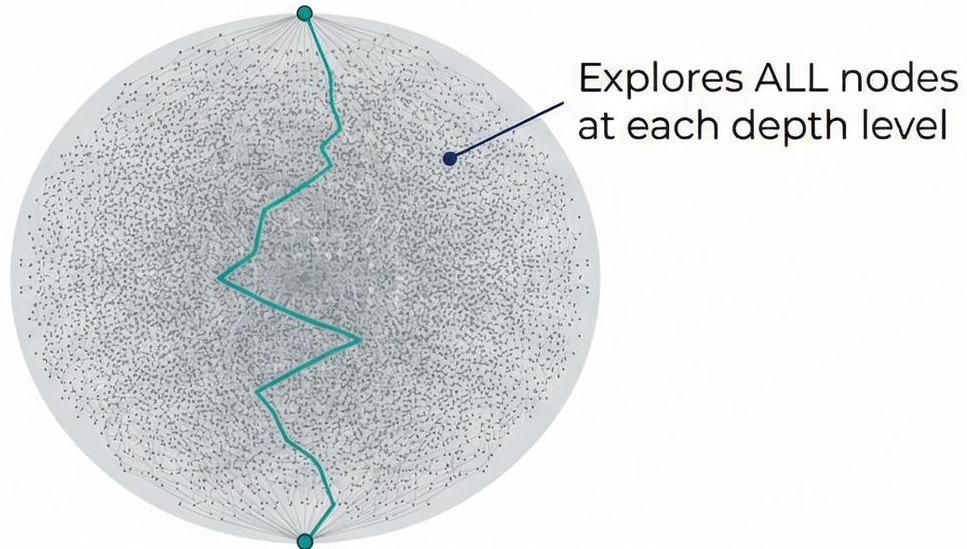
Legend:

-  = Misplaced tile (wrong position)
-  = Correct tile / Blank tile
-  = Selected node (lowest f)



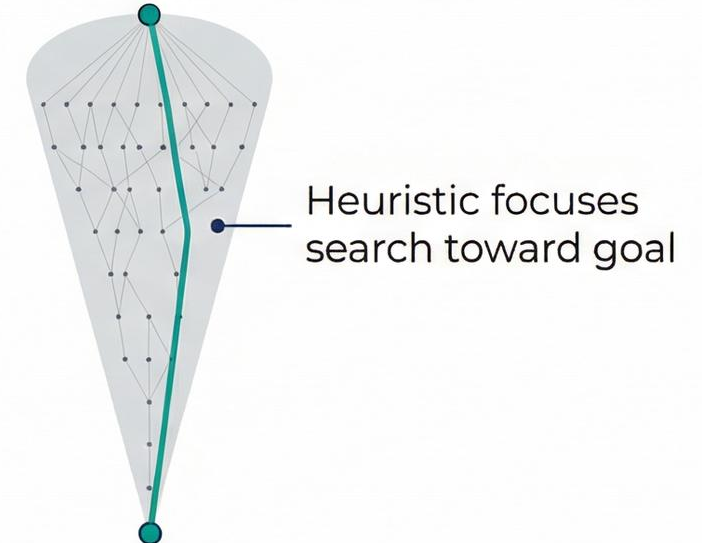
BFS vs Heuristic Search on 8-Puzzle

BFS (Breadth-First Search)

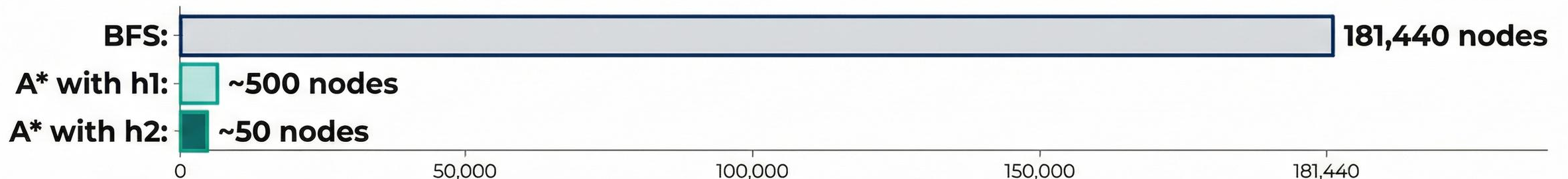


Nodes expanded: ~181,000+ for depth 20

A* with h1 (Misplaced Tiles)



Nodes expanded: ~500 for same problem



Heuristic search explores a tiny fraction of the state space!



Informedness: h2 Dominates h1

Definition: h2 DOMINATES h1 if $h2(n) \geq h1(n)$ for ALL states n
A more informed heuristic = fewer nodes expanded

7	2	4
5	—	6
8	3	1

h1 (Misplaced) = **6**

h2 (Manhattan) = **14**

True cost **h*** = 22 moves



h2 is **CLOSER** to **h*** => more informed

Average Nodes Expanded (100 random instances)

Depth	h1 nodes	h2 nodes
4	13	12
8	39	25
12	227	73
16	1301	211
20	7276	676
24	39135	1641

Always prefer a **MORE INFORMED** heuristic (higher **h** values)
As long as it remains **ADMISSIBLE** (never overestimates)

Effective Branching Factor

b^* = effective branching factor

If A^* expands N nodes at depth d :

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

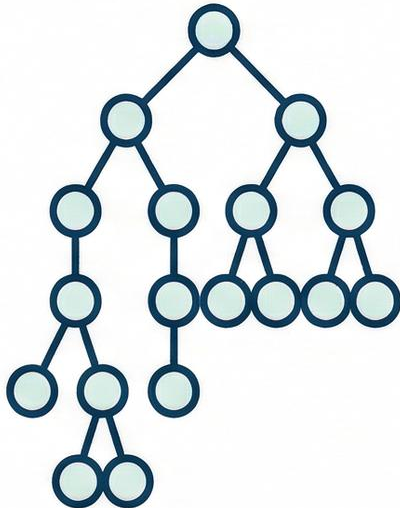
A good heuristic has **b^*** close to **1.0**

$b^*=1.0$



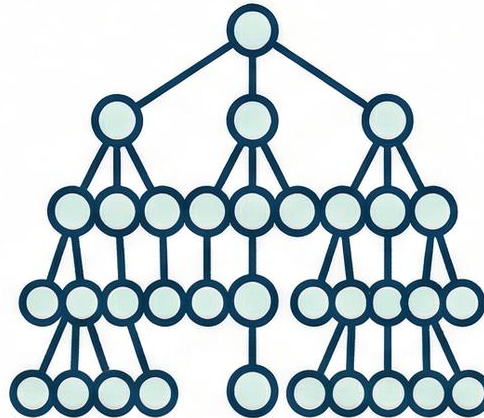
Perfect heuristic

$b^*=1.5$



Good heuristic

$b^*=3.0$



No heuristic (BFS)

Effective Branching Factor for 8-Puzzle

Depth	h1 (b^*)	h2 (b^*)
4	1.45	1.13
8	1.33	1.25
12	1.42	1.24
16	1.45	1.25
20	1.47	1.27
24	1.48	1.26
24	1.48	1.26

h2 consistently has **LOWER** b^* (closer to 1.0)

h2 is the **BETTER** heuristic for 8-puzzle

Quality of heuristic = how close b^* is to 1.0
 $b^* = 1.0$ means the heuristic is perfect

Why A*? Greedy vs UCS

UCS



- Looks backward only
- Uses $g(n)$ = cost so far
- Complete and Optimal
- But: Explores too many nodes

Safe but slow

Greedy



- Looks forward only
- Uses $h(n)$ = estimate to goal
- Not Complete, Not Optimal
- But: Very fast when it works

Fast but risky

A*



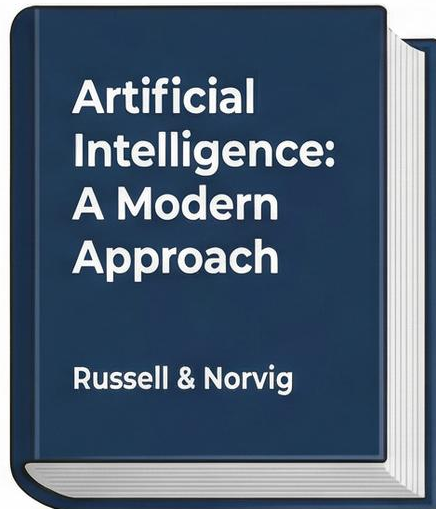
- Looks both ways!
- Uses $f(n) = g(n) + h(n)$
- Complete and Optimal (if h admissible)
- Best of both worlds

Fast AND safe

A* combines the strengths of both!



Textbook Reference



Chapter 3.5 - 3.6

- **Section 3.5:** Informed (Heuristic) Search Strategies
 - Greedy Best-First Search
 - A* Search
- **Section 3.6:** Heuristic Functions
 - Effect of heuristic accuracy
 - Generating admissible heuristics

Key Topics Covered

- ✓ Informed vs Uninformed Search
- ✓ Heuristic Functions $h(n)$
- ✓ Greedy Best-First Search
- ✓ A* Search Algorithm
- ✓ Admissibility
- ✓ Dominance & Informedness
- ✓ Effective Branching Factor

Next Session: Adversarial Search (Game Playing)